Where a GUI attaches to the Zero Trust Backplane

Design note for the ZT Console (`packages/zt-console`): the attachment points, what was built on them, the rules it follows, its limitations and what remains in the terminal. For how to *use* the GUI see the GUI user guide.

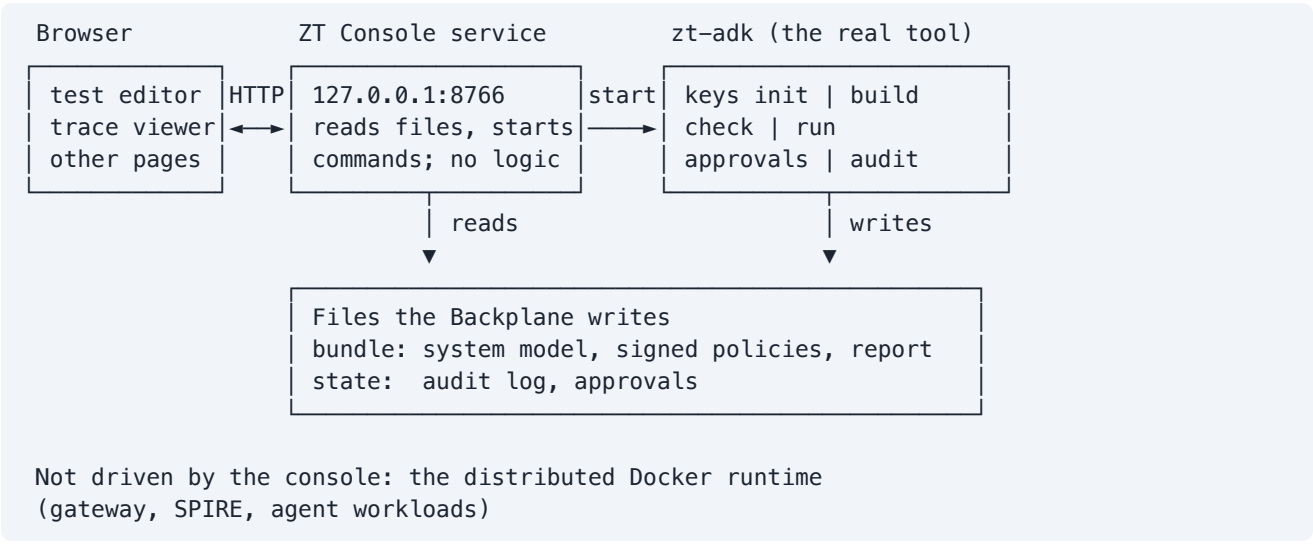
Question	Where can a GUI be added to the Backplane so people can understand the tool and run tests on it?
Short answer	On top of what already exists. The Backplane is a set of commands that write files. A GUI needs no change to the protection code: it starts the same commands and reads the same files.
Status	Built as the new package `zt-console`: a test editor and trace viewer plus Dashboard, Learn, System, Policies, Audit and Approvals pages. **No code or configuration file of the repository was changed**; the root `README.md` only gained documentation of the GUI.

1. Starting point

The Backplane has command-line tools (`zt-adk`, `zt-distributed`, `zt-psl`, `zt-scan`) and no web assets. Everything it produces is a file: the signed bundle (system model, bindings, policy store, report), the audit log, the approval store and, for the distributed runtime, event and span logs. Files this stable and commands this complete are a good base for a GUI.

The design rule is that **the GUI holds no protection logic**. It starts the real command and reads the real files, so there is no second code path that can drift from the tool.

2. How it fits together



The service listens on the local machine only. Every run, build or check is the real `zt-adk` command started with an argument list (no shell), one at a time, with a time limit. What the page shows is read from the bundle

and the audit log.

3. Attachment points

The user wants to	Attaches to	How the console uses it
Run any message, several turns, a chosen role	`zt-adk run --role=R -- "message" ...`	Starts the command; the records it appends to the audit log are shown as the trace
See what each check decided	`state/audit.jsonl`	Records written during the run are read and explained in plain language
Say what should happen and get pass or fail	Console side (`evaluate`)	Outcome, boundary, rule that must fire, text the answer must or must not contain
Set up an application	`zt-adk keys init`, `build`, `check`	The same commands, started from buttons; output shown
Stop a run or a long build	The `zt-adk` process the console started	Ends the command and the processes it started; records written before the stop remain
Read the rules	`policy_store/*.psl` annotations (`@rule`, `@nl`, `@permit`, `@on_violation`)	Parsed into a searchable list; rule ids in traces show the rule's own sentence
See the application	`system_model.json`, `bindings.json`	Agent tree by domain, tools with classification evidence, routes
Approve a held operation	`approvals/requests` and `decisions`, `zt_adk.approvals`	Lists requests with the exact arguments; signing only in developer mode
Check the audit trail	Hash chain in `audit.jsonl`	Same algorithm as `zt-adk audit verify`, with a note that the chain is unkeyed
Re-run checks after a change	Saved tests and run history (the console's data folder)	Batches and campaigns; results export as JSON, CSV or Markdown

4. What was built

Page	What it does
Dashboard	First-run checklist, decisions by boundary, the rules that stopped or held requests, recent runs, and a button that runs the demo suite.
Test Lab	Test editor and trace viewer. Any message, several turns, role, expected outcome; saved tests, batches and campaigns; run history; export.
Learn	The boundaries, the life of a request, a glossary and the command-line equivalent of every button.
System	Agents by security domain, every tool with how it was classified and why, every route.
Policies	Every rule in plain language, searchable.
Audit	Every decision with a plain reading, filters and chain verification.

Page	What it does
Approvals	Held operations with the exact arguments being approved.

Start it with `zt-console --profile <zt_adk.yaml>` (details in the user guide).

5. Footprint: why the released code is unaffected

- The GUI is **one new directory**, `packages/zt-console`. No tracked source or configuration file is modified: not the other packages, the changelog, the CI workflow or the root `.gitignore`. The only existing file touched is the root `README.md`, which gained documentation of the GUI (section 2.9, a note under Tests and a line in the layout).
- Nothing imports it. The Backplane packages do not depend on it; it depends on the Backplane only through the `zt-adk` command and, for developer-mode approvals only, three lazily imported modules (`zt_adk.approvals`, `zt_adk.keys`, `zt_pdp.evidence_issuer`).
- The existing CI runs a fixed list of test suites that does not include `zt-console`, so CI behaviour is unchanged.
- Removing the package (`pip uninstall zt-console`, delete the folder) and reverting the README additions returns the repository to exactly its previous state.
- The console writes only to its own data folder (git-ignoring itself) and to the `.zt` folder next to the profile it is told to use, the same place `zt-adk` itself writes.

6. Rules the GUI follows

The console follows these rules.

Rule	Why
Never hold approver keys on the server	Approvals are meant to be signed on approver devices. The console shows requests, and signs only in developer mode (`--dev-approve`) with local keys.
Treat every displayed string as hostile	Messages, tool arguments and audit records can contain markup. Render as text only, under a strict content security policy.
Do not let the page choose paths or commands	The console accepts only the profiles it was started with and passes messages as arguments, never through a shell.
Authenticate users before exposing it beyond localhost	The tool has no end-user authentication; neither does the console today.
Say what the audit check proves	The chain is unkeyed; the page says it detects accidental or naive edits only.
Show the active mode prominently	In shadow mode, denied non-critical operations still run.

7. Limitations and what stays in the terminal

What stays in the terminal. The GUI does not do the following. The commands are the Backplane's own (repository README, sections 2 to 5):

Task	Terminal
Create or edit the profile	`zt-adk init --app "module:root_agent"`, then edit `zt_adk.yaml`
Write or change policy rules	The policy files (PSL; README section 4), then `zt-adk build`
Add an approver once the keys exist	`zt-adk keys add-approver NAME [--secondary]`, then rebuild
Pass build options, for example skip the static scan	`zt-adk build --no-static`
Inspect the discovered system as JSON	`zt-adk inspect --json`
Approve operations in production, with approver keys on the approvers' devices	`zt-adk approvals approve ID --approver NAME [--secondary]`
Run and manage the distributed Docker runtime	`zt-distributed build`, `prepare`, `start`, `run`, `approvals`, `admin`, `verify`, `stop` (README section 5)

Limitations of the GUI:

- It drives the in-process protection (`zt-adk run`) only.
- It has no user login and is meant for one local user; it listens on the local machine only.
- It reads answers from the command's text output and attributes audit records to a run by time.
- Its audit-chain check is unkeyed, like `zt-adk audit verify`.
- It was tested with the scripted model on the example applications (no live model), on macOS with Chrome.

The complete list is in section 7 of the user guide.